Recursion Trees and the Master Theorem

Prof. Timothy Sun

Recursion

Recursive algorithms have recursive descriptions of their runtimes:

```
MergeSort(A[1,...,n]):
    if n > 1
        m := ⌊n/2⌋
        MergeSort(A[1,...,m])
        MergeSort(A[m+1,...,n])
        Merge(A,m)

Merge(A[1,...,n],m):
    initialize B[1,...,n]
    i := 1
    j := m+1
    for k = 1,...,n
        //pick smallest element blah blah...
    for k = 1,...,n
        A[k] := B[k]
```

Recursion

Recursive algorithms have recursive descriptions of their runtimes:

- Mergesort makes two recursive calls on arrays of size ≈ $\boldsymbol{n}/2$
- The rest of the alg. is O($\boldsymbol{n}$)

```
MergeSort(A[1,...,n]):
    if n > 1
        m := ⌊n/2⌋
        MergeSort(A[1,...,m])
        MergeSort(A[m+1,...,n])
        Merge(A,m)

Merge(A[1,...,n],m):
    initialize B[1,...,n]
    i := 1
    j := m+1
    for k = 1,...,n
        //pick smallest element blah blah...
    for k = 1,...,n
        A[k] := B[k]
```

Recursion

Recursive algorithms have recursive descriptions of their runtimes:

- Mergesort makes two recursive calls on arrays of size ≈ $n/2$
- The rest of the alg. is $\Theta(n)$

We need to solve the recurrence

$$T(n) = 2T(n/2) + \Theta(n).$$

```
MergeSort(A[1,...,n]):
    if n > 1
        m := ⌊n/2⌋
        MergeSort(A[1,...,m])
        MergeSort(A[m+1,...,n])
        Merge(A,m)

Merge(A[1,...,n],m):
    initialize B[1,...,n]
    i := 1
    j := m + 1
    for k = 1,...,n
        //pick smallest element blah blah...
    for k = 1,...,n
        A[k] := B[k]
```

Recursion

Substitution is kind of hard to understand…

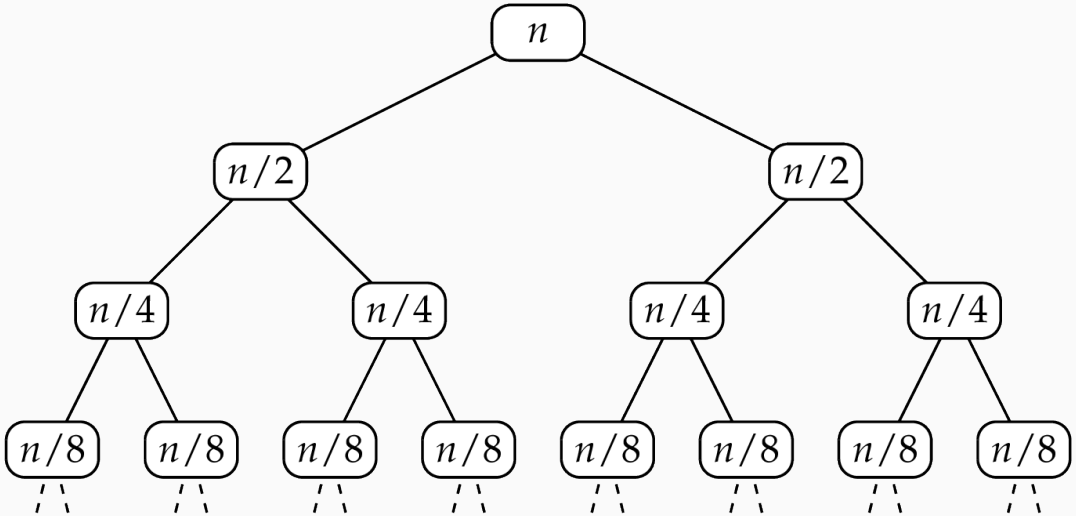
$T(\boldsymbol{n}) \quad = 2T(\boldsymbol{n}/2) + c\boldsymbol{n}$

Recursion

Substitution is kind of hard to understand…

$$
\begin{aligned}
T(n) &= 2T(n/2) + cn \\
&= 2(2T(n/4)+cn/2) + cn \\
&= 2(2(2T(n/8)+cn/4)+cn/2) + cn \\
&\ldots \\
&= \ldots T(1) \ldots
\end{aligned}
$$

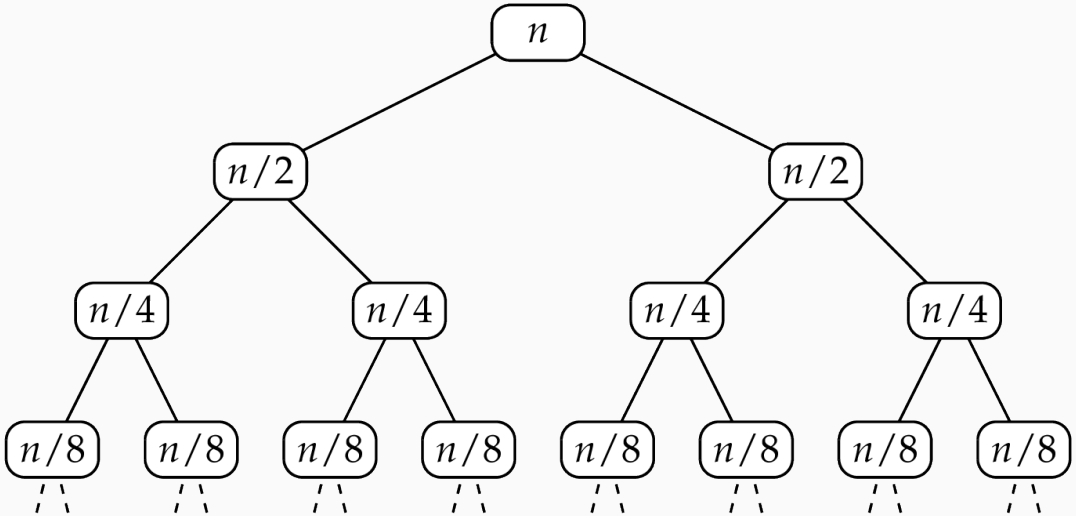
Recursion trees



Recursion trees

Total time:

T(***n***) = sum of all the nodes

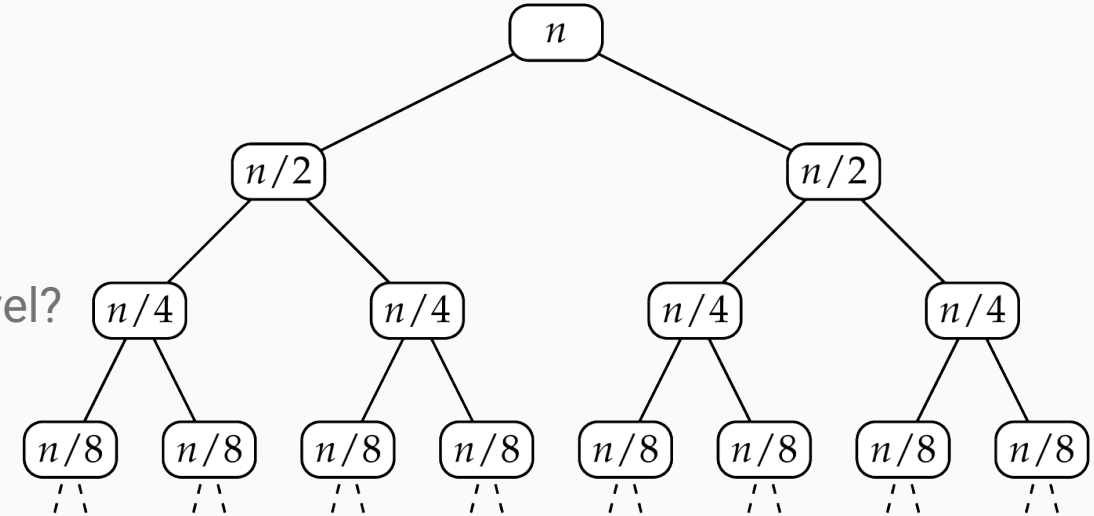


Recursion trees

Total time:

T(***n***) = sum of all the nodes

- What's the sum at each level?

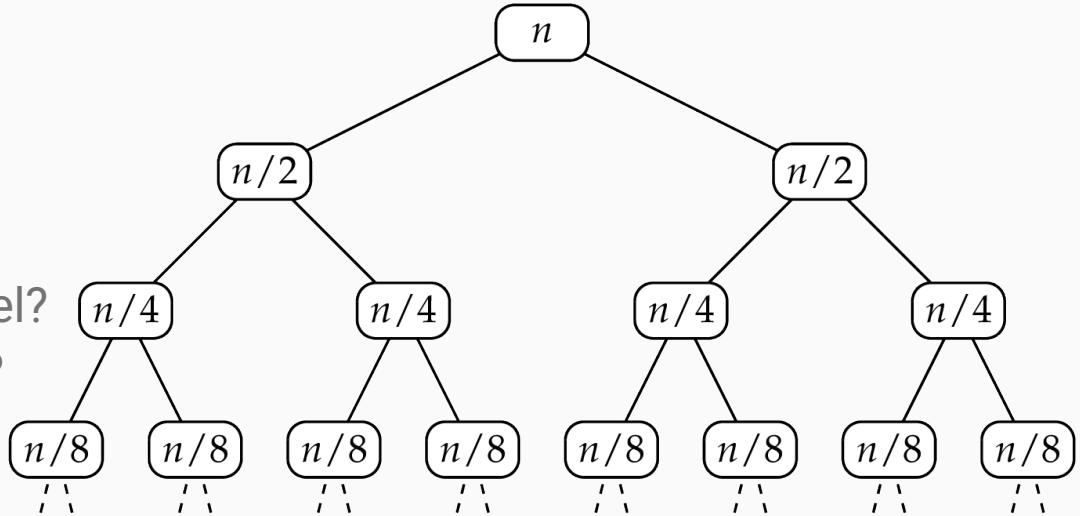


Recursion trees

Total time:

T(***n***) = sum of all the nodes

- What's the sum at each level?
- How many levels are there?

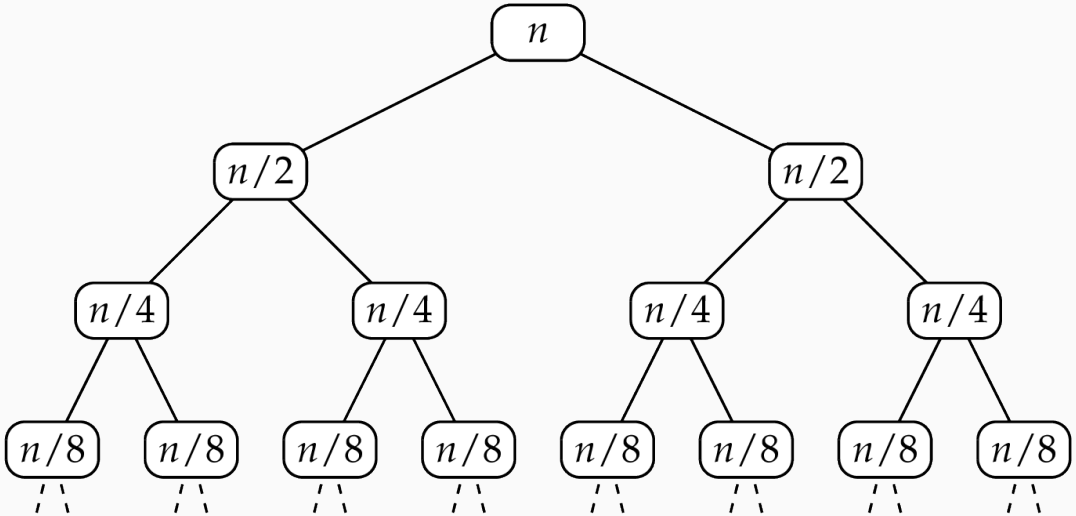


Recursion trees

$\Theta(\boldsymbol{n})$ work per level,

$\Theta(\log \boldsymbol{n})$ levels

$T(n) = \Theta(\boldsymbol{n} \log \boldsymbol{n})$



Another example

Consider the following (useless) algorithm:

```
TreeEx(n)
    if n > 1
        TreeEx(n/4)
        x := 0
        for i = 1, ..., n
            for j = i, ..., n
                x := x + 1
        TreeEx(n/4)
        TreeEx(n/4)
```

Another example

Consider the following (useless) algorithm:

$\textsc{TreeEx}(n)$
  if $n > 1$
    $\textsc{TreeEx}(n/4)$
    $x := 0$
    for $i = 1, \ldots, n$
      for $j = i, \ldots, n$
        $x := x + 1$
    $\textsc{TreeEx}(n/4)$
    $\textsc{TreeEx}(n/4)$

3 recursive calls

Another example

Consider the following (useless) algorithm:

$\textsc{TreeEx}(n)$
  if $n > 1$
    $\textsc{TreeEx}(n/4)$
    $x := 0$
    for $i = 1, \ldots, n$
      for $j = i, \ldots, n$
        $x := x + 1$
    $\textsc{TreeEx}(n/4)$
    $\textsc{TreeEx}(n/4)$

3 recursive calls

everything else $\Theta(n^2)$

Another example

Consider the following (useless) algorithm:

$\textsc{TreeEx}(n)$
  if $n > 1$
    $\textsc{TreeEx}(n/4)$
    $x := 0$
    for $i = 1, \ldots, n$
      for $j = i, \ldots, n$
        $x := x + 1$
    $\textsc{TreeEx}(n/4)$
    $\textsc{TreeEx}(n/4)$

3 recursive calls

everything else $\Theta(n^2)$

$$T(n) = 3\,T(n/4) + \Theta(n^2)$$

Solving the recursion tree

$$T(n) = 3\,T(n/4) + \Theta(n^2)$$

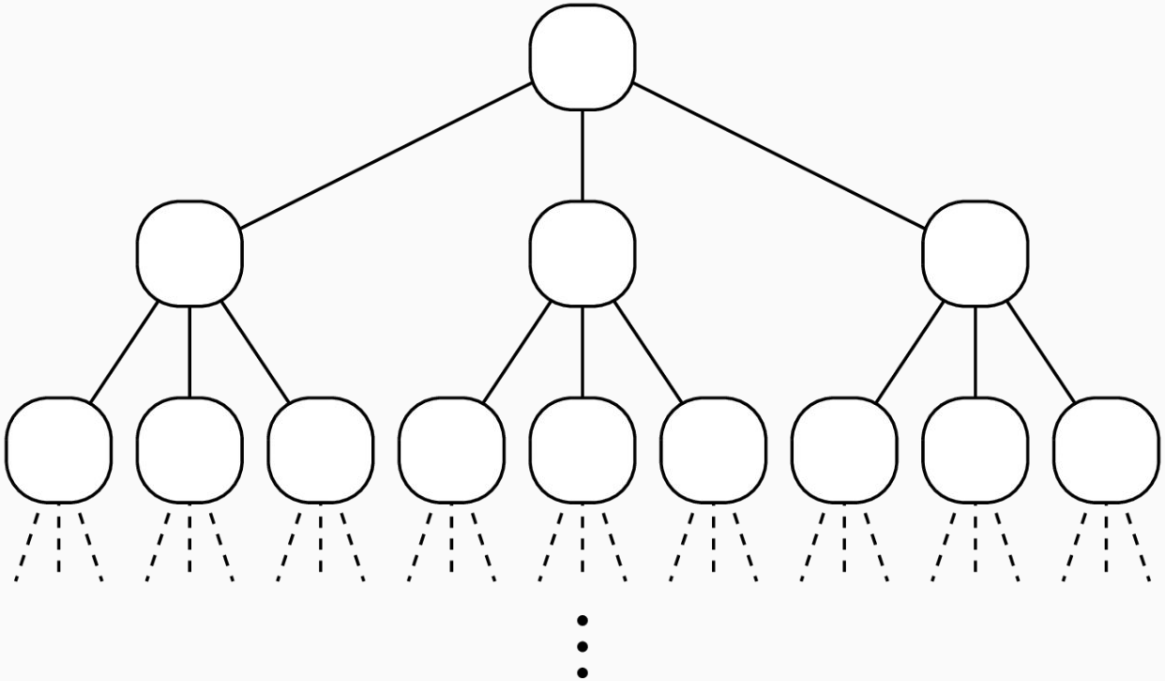
Solving the recursion tree

$$T(m) = 3\,T(m/4) + \Theta(m^2)$$

I like to use a new variable so I know exactly what to substitute

Solving the recursion tree

$$T(m) = \boxed{3}\,T(m/4) + \Theta(m^2)$$



Since the function has 3 recursive calls, each node has 3 children

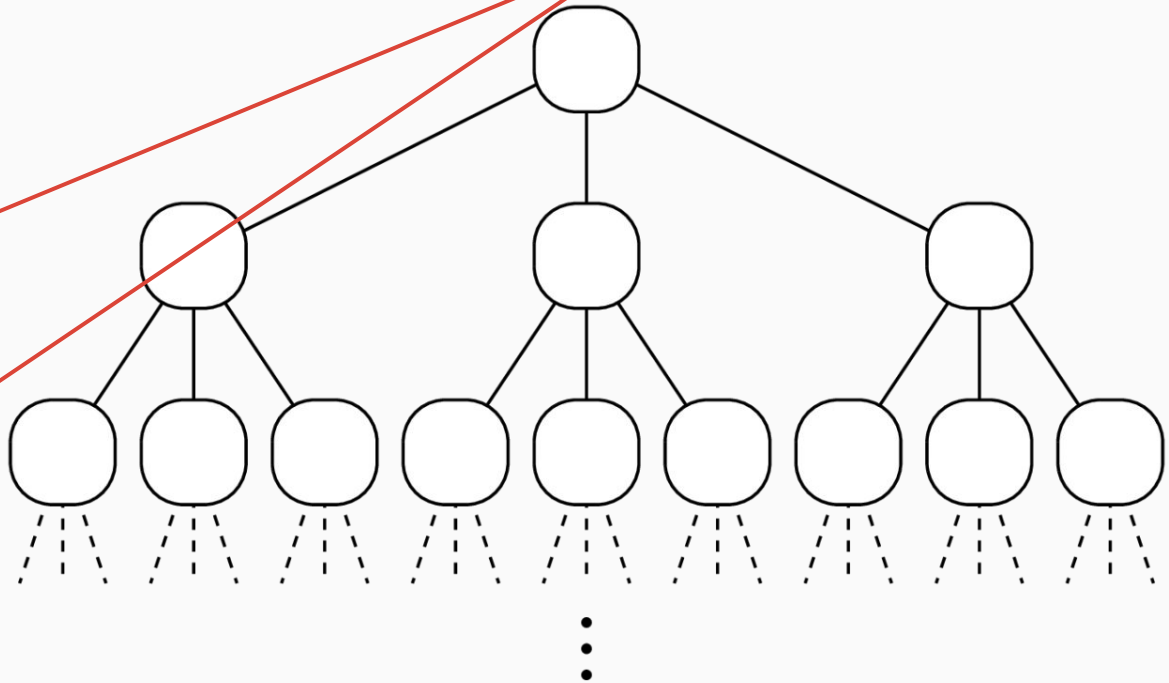
Solving the recursion tree

$$T(m) = 3\,T(m/4) + \Theta(m^2)$$

$m = n$

$m = \frac{n}{4}$

$m = \frac{n}{16}$



Get next input size by plugging in current input size

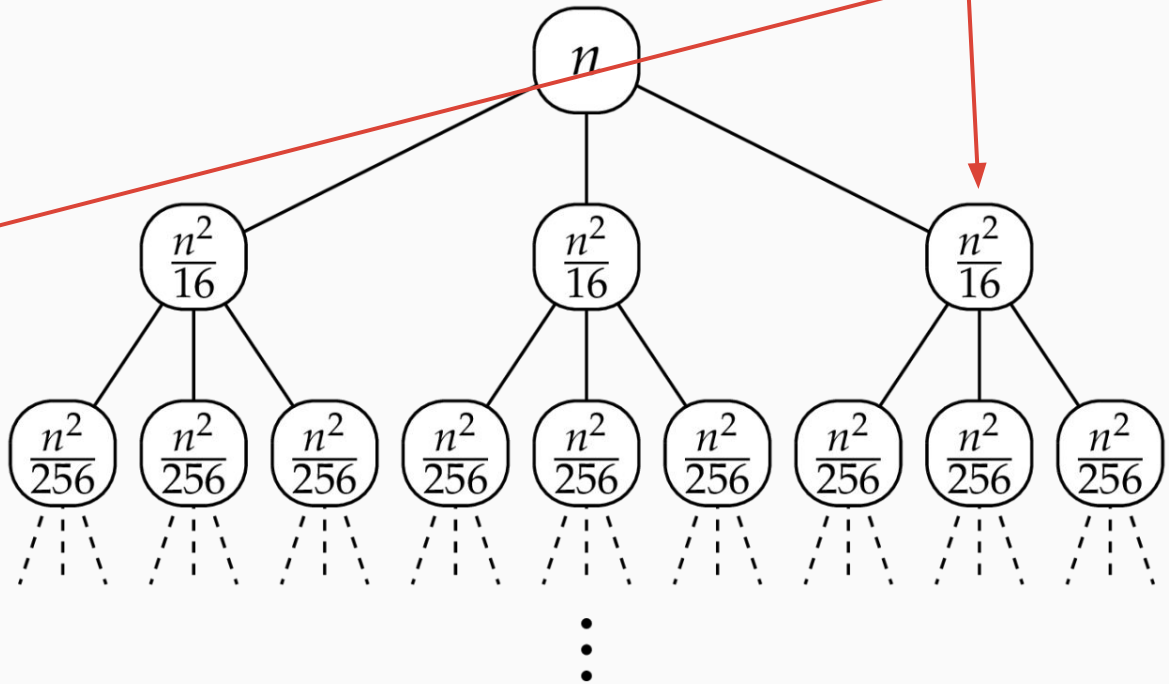
Solving the recursion tree

$$T(m) = 3\,T(m/4) + \Theta(m^2)$$

$m = n$

$m = \frac{n}{4}$

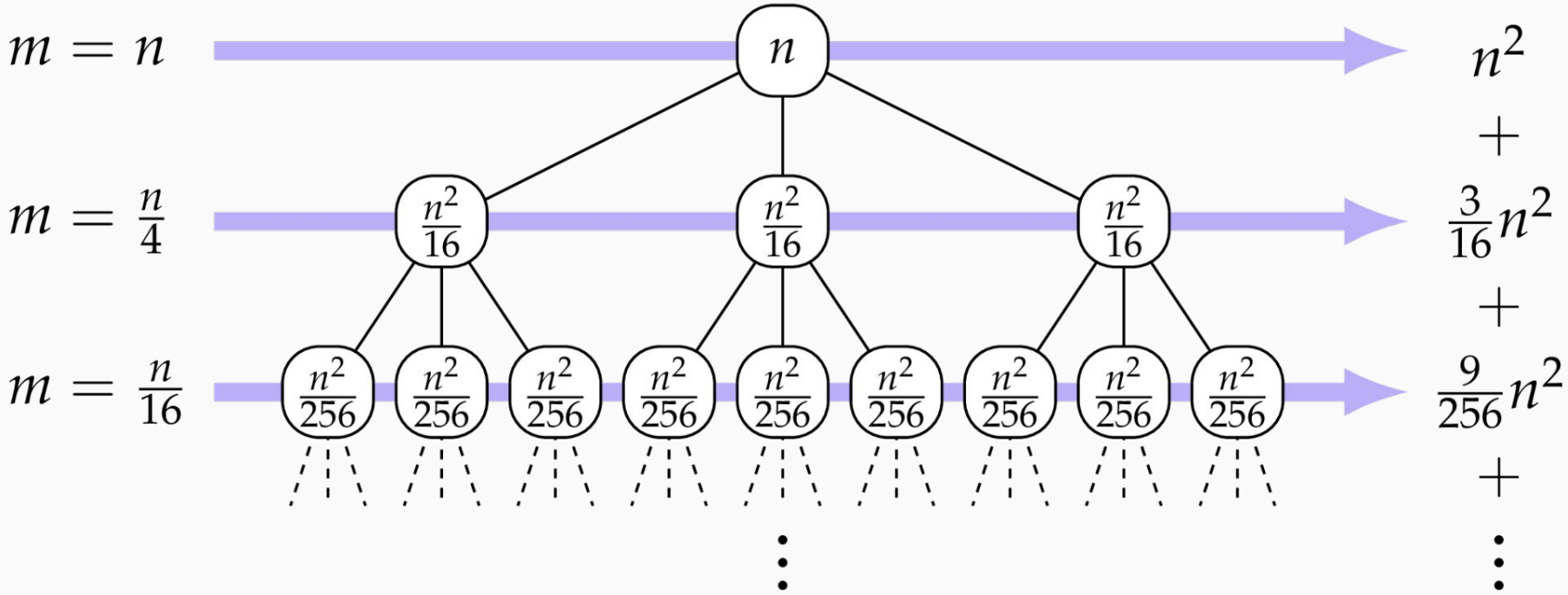
$m = \frac{n}{16}$



Get node labels by plugging in the input size

Solving the recursion tree

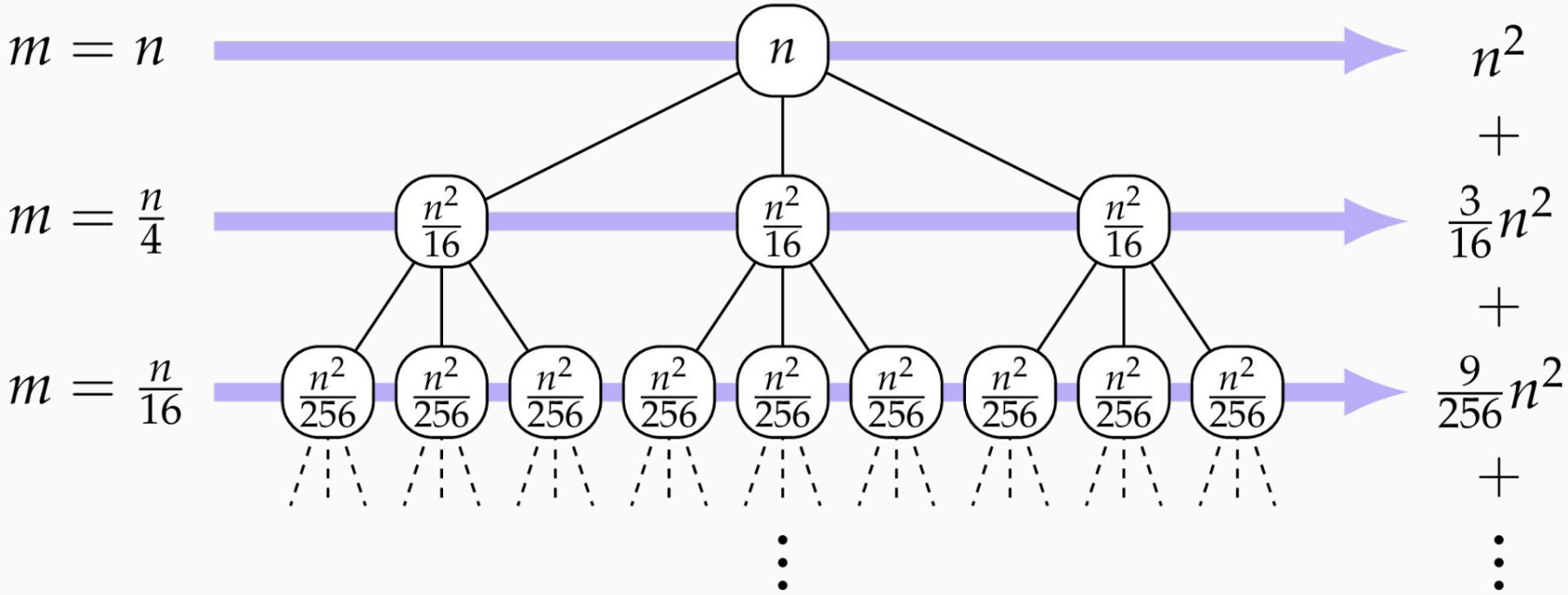
$$T(m) = 3\,T(m/4) + \Theta(m^2)$$



Compute “level sums” by adding up nodes in each layer

Solving the recursion tree

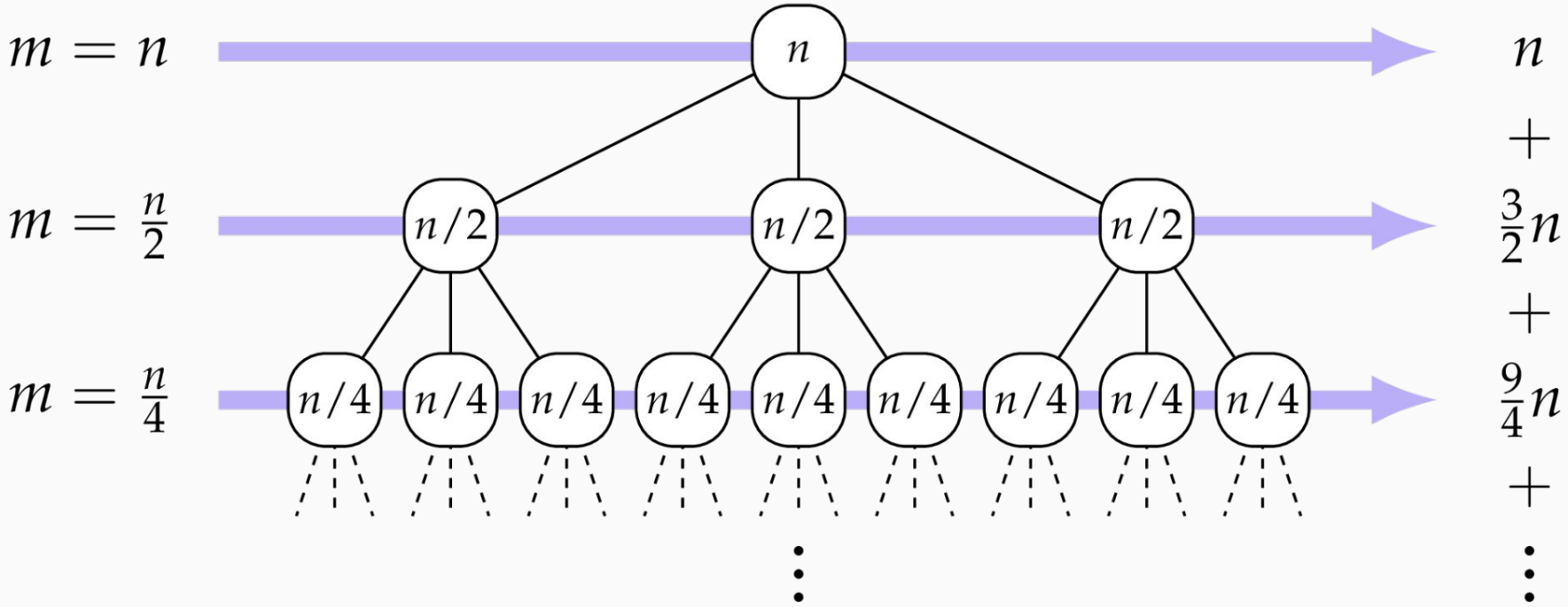
$$T(m) = 3\,T(m/4) + \Theta(m^2)$$



A ***root-heavy*** tree, so $T(n) = \Theta(n^2)$

Other types of trees

$$T(n) = 3\,T(n/2) + \Theta(n) = \Theta(n^{\log_2 3})$$



A ***leaf-heavy*** tree

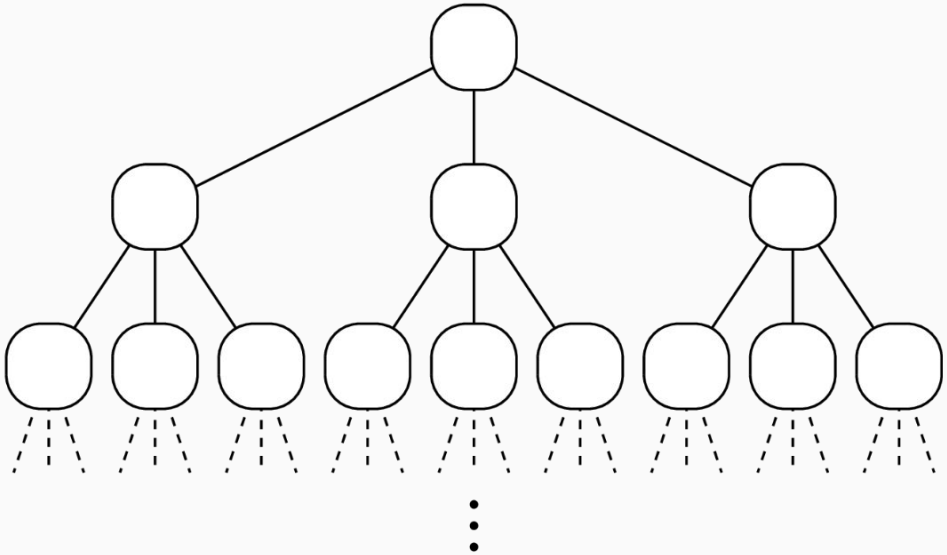
Counting leaf nodes for leaf-heavy trees

The number of leaves is determined by ***number of children*** per node and ***depth***

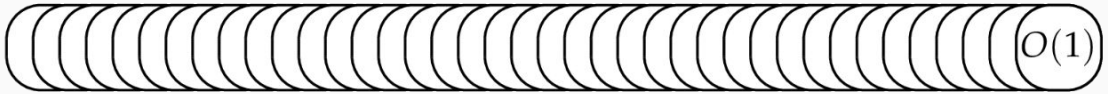
$\ell = 0$ $\quad 3^0 = 1$

$\ell = 1$ $\quad 3^1 = 3$

$\ell = 2$ $\quad 3^2 = 9$

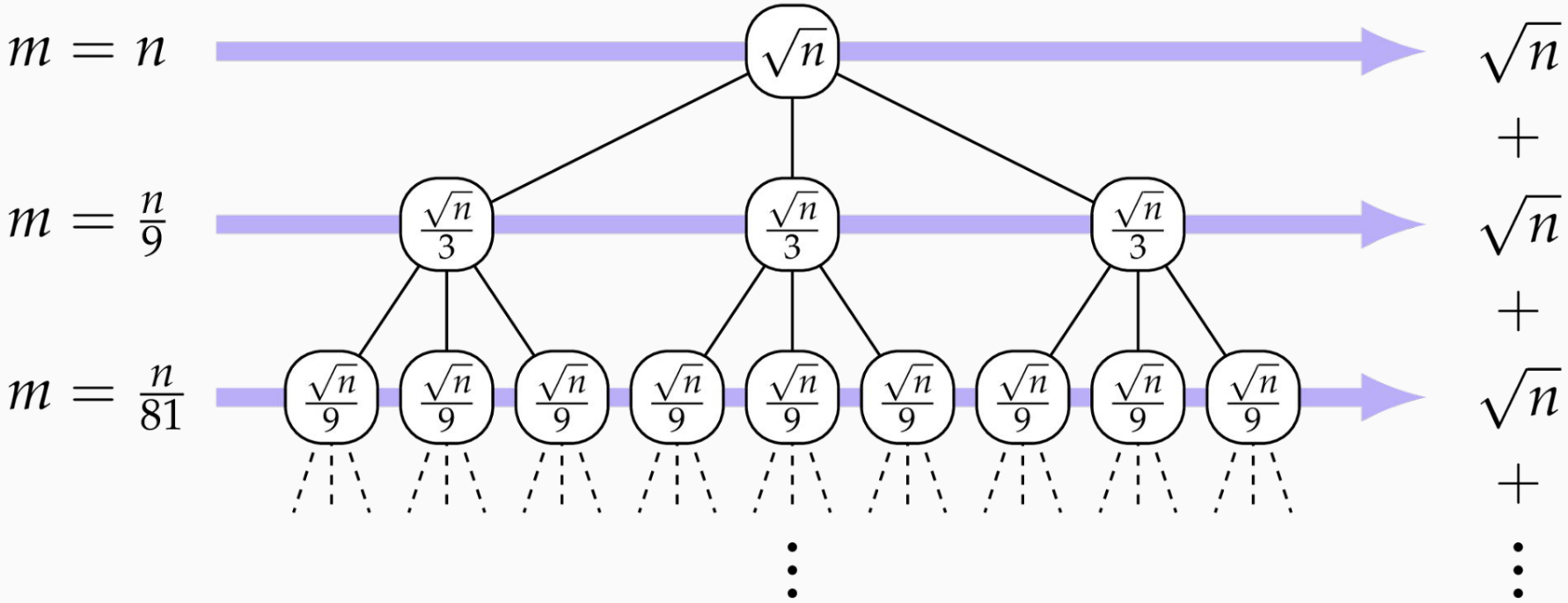


$\ell = \log_2 n$ $\quad 3^{\log_2 n} = n^{\log_2 3}$



Other types of trees

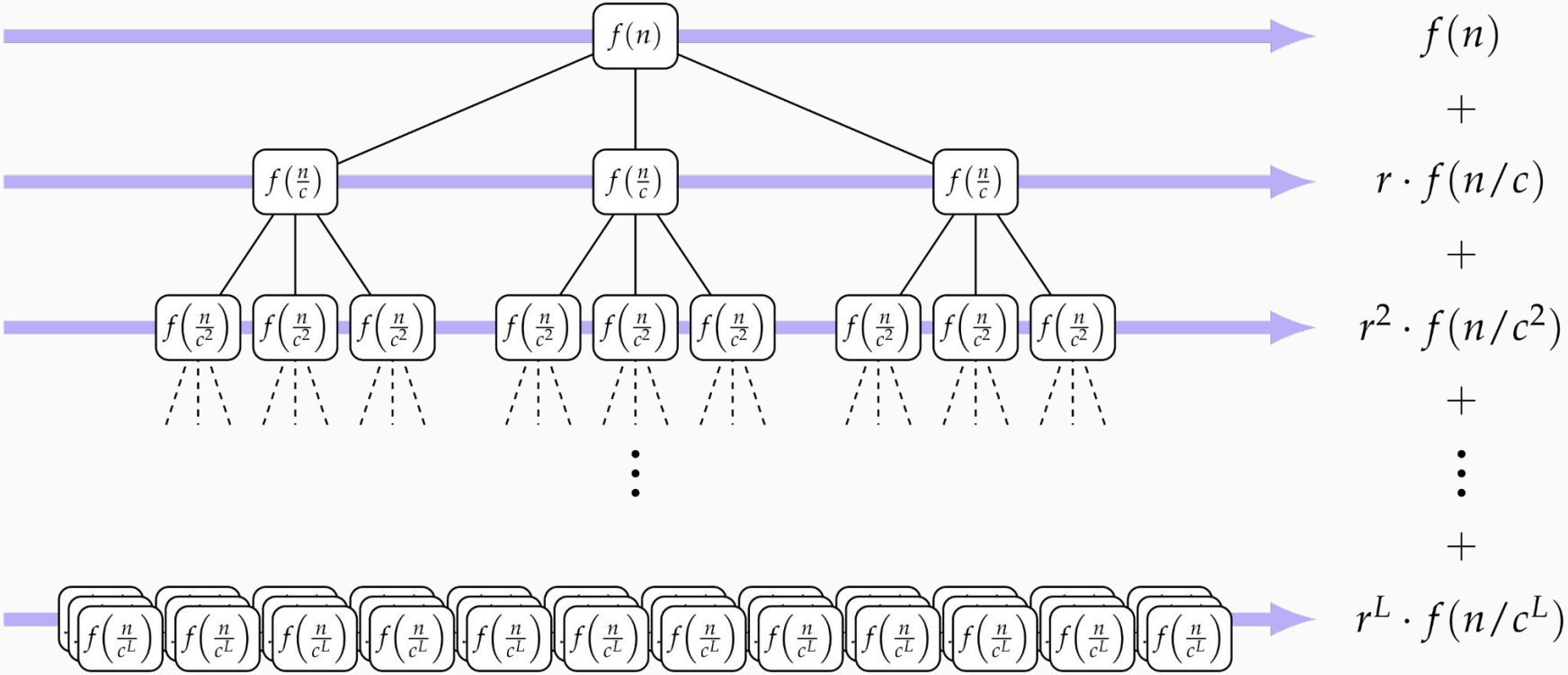
$$T(n) = 3\,T(n/9) + \Theta(\sqrt{n}) = \Theta(\sqrt{n}\log n)$$



A ***balanced*** tree

Recursion trees

$$T(n) = r\,T(n/c) + \Theta(n^a)$$

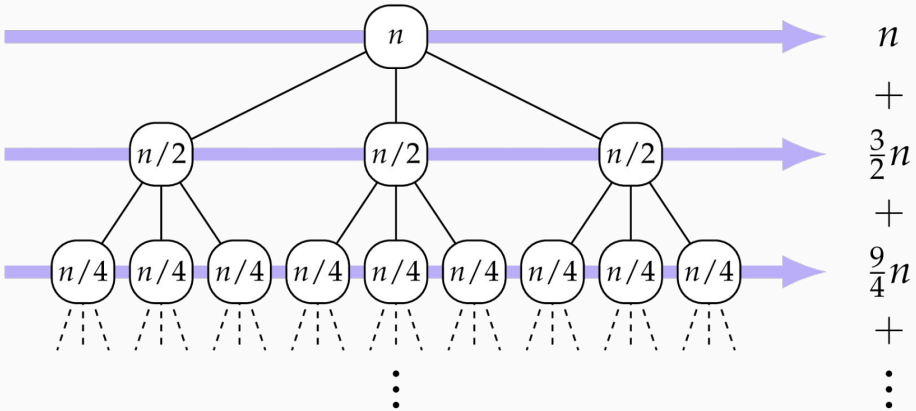


Recursion trees

General themes:

$$T(n) = r\,T(n/c) + \Theta(n^a)$$

- Level sums form geometric series
- Biggest term dominates
  - First term? O(root)
  - Last term? O(# leaves)
- Balanced: # levels * root



Geometric series

If ***r*** < 1, then:

$$a + ar + ar^2 + ar^3 + \ldots = \sum_{i=0}^{\infty} ar^i = \frac{a}{1-r} = O(a)$$

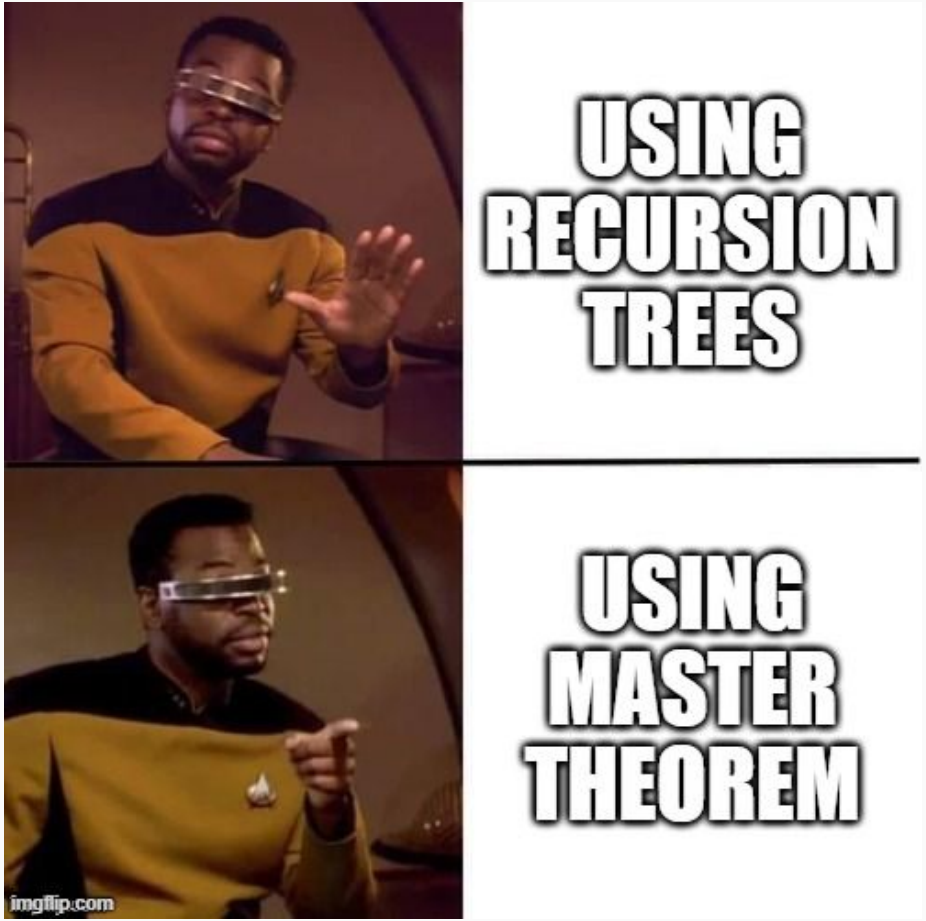
Master theorem

$$T(n) = r\,T(n/c) + \Theta(n^a)$$

Let ***b*** = $\log_c$ ***r***. Then one of these three cases holds:

a vs. b	$T(n)$
$<$	$\Theta(n^b)$
$=$	$\Theta(n^a \log n) = \Theta(n^b \log n)$
$>$	$\Theta(n^a)$

When possible, use the master theorem!



based on a meme by Arielle R.-T. (F'22)

Non-master recurrences

$$A(n) = A(n/3) + A(2n/3) + \Theta(n)$$
$$B(n) = B(n-1) + B(1) + \Theta(n)$$
$$C(n) = 2\,C(n-1) + \Theta(1)$$

Can sometimes still use recursion trees, but might need other methods.